

Supplementary Material

Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows

Edward Y. Chang
Stanford University

Longling Geng
Stanford University

This document is the supplementary material for the paper *Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows*. It contains the proofs, full result tables, engineering specifications, and worked examples referenced by the main paper. Theorems, sections, and figures of the main paper are cited by name; items defined here are numbered with an “S” prefix (e.g., Table S1).

Contents

S1	Proofs of the Safety Theorems	2
S2	Expanded Failure Model	4
S3	Additional Safety Invariants	4
S4	Benchmark Roster	5
S5	Aggregate Safety Scoreboard	5
S6	Per-Proposer-Class Detail	6
S7	Concurrency, Contention, and Distributed Execution	6
S8	Per-Property Safety Results	7
S9	Live-Proposer Pilots (RQ3): Detail and Representative Cases	7
S10	Coverage Audit: ATP and a Production Transaction-Processing Substrate	9
S11	RQ5 Infrastructure-Cost Audit: Full Method and Results	10
S12	R8 Deployment Smoke-Load Variability	11
S13	Mnemosyne Architecture and Runtime Data Structures	12
S14	Admission, Commit, Logging, and Replay	15
S15	Disruption, Recovery, and Compensation	16

S16	Staged Implementation (R0–R8) and Reproducibility	18
S17	Structure and Scalability of the Constraint Set $\mathcal{C}$	19
S18	Workflow-Engine and Storage-Substrate Selection	20
S19	Worked Proposal Examples	22
S20	An End-to-End Transaction Trace	24

S1 Proofs of the Safety Theorems

We restate, by name, the four safety theorems of the main paper and give their proofs. All claims are relative to the trusted, declared constraint set $\mathcal{C}$.

Authority-Separation Theorem. We prove the claim by induction on the length of CTL. Initially, CTL is empty or contains only trusted initial records, so every committed transition is $\mathcal{C}$ -valid. Assume after k appends that every transition in CTL_k is $\mathcal{C}$ -valid. Consider the $(k+1)$ -st committed transition. By assumption, no LLM, solver, agent, ACR wakeup, learned policy, or adapter can append to CTL directly; such components can only emit a proposal p . Since the only operation that extends CTL is the append of an admitted transition, p can become the next committed transition only if the gate admits it under $\mathcal{C}$ and the current effective state. By the soundness of the gate with respect to $\mathcal{C}$, the appended transition is $\mathcal{C}$ -valid. Thus every transition in CTL_{k+1} is $\mathcal{C}$ -valid. By induction, all committed transitions are $\mathcal{C}$ -valid. The argument never assumes that the intelligence layer is accurate, rational, complete, honest, or improving; therefore committed-state correctness is decoupled from that layer. $\square$

Serial-Equivalent Generative Admission Theorem. For each conflict scope, admission imposes a total order on accepted proposals: if two proposals contend for the same scope, the gate admits at most one before the other observes the updated effective state or is rejected as conflicting. Because append is atomic, each admitted proposal contributes one indivisible committed transition. Interleavings across disjoint scopes commute with respect to the observations made through `StateView`, so the global committed history is equivalent to an interleaving of the per-scope serial orders. A dependent recovery proposal is evaluated only after its dependencies appear in CTL and after `StateView` reflects the effective projection of committed records. Rejected or in-flight proposals never appear in this projection. Thus each recovery step can be explained by a serial prefix of committed history, and recovery decisions are independent of speculative attempts that never committed. $\square$

Evidence-Preserving Repair Safety Theorem. The trigger is evaluated over committed evidence in CTL and effective evidence in `StateView`. Suppose a repair r is admitted and discharges the trigger. If r leaves E effective and queryable, then the trigger has not been discharged by destroying evidence. Otherwise, r deletes, compensates, or obscures E . By the evidence-preservation invariant, such a repair is admissible only when the gate verifies that the underlying failure condition F is resolved under $\mathcal{C}$. Hence no admitted repair can succeed solely by removing the observable signal of F : it either preserves the evidence or resolves the condition. Since CTL is append-only, the evidence and the gate decision remain auditable after the fact. $\square$

Obligation Containment Theorem. Creation of an ACR is already a committed transition, so its existence is governed by ordinary admission. Firing an ACR is not a commit operation; it resumes

a continuation that constructs a proposal package. By assumption, that package must pass ordinary admission before any domain transition can be appended. Therefore an ACR has no direct mutation path to CTL or StateView. It can influence committed truth only by generating a proposal that the same gate admits for every other proposer. A future obligation can wake, but it cannot corrupt. $\square$

Proofs of the Reactive-Repair Propositions (main paper, Section 3)

Proposition (Bounded Reactive Repair).

Proof. Each repair round validates the affected region in $\mathcal{O}(JM)$ and propagates effects along effective dependencies in $\mathcal{O}(JO_{\max})$; rounds are capped at K , giving the time bound. The displacement budget is checked each round and escalation triggers as soon as it is exceeded, so cumulative displacement never passes $\delta_{\max}$. Commit occurs only on an empty violation set (admission), so no infeasible transition is appended; if no in-budget repair is found within K rounds, rollback restores the last validated state. $\square$

Proposition (Transactional Rollback Safety).

Proof. Restore points are created only after admission (empty violation set), and committed-transition-log entries are immutable; superseded branches are excluded from the effective StateView but retained for audit. Compensation handlers keyed by idempotency keys logically undo effects after S_j under the dependency-closure rule, so rollback returns to a previously admitted, $\mathcal{C}$ -feasible state from which replay is deterministic. $\square$

Proposition (Recursive recovery reduces to sequential recovery).

Proof. By Lemma ?? a candidate depends only on $(S_k, \mathcal{C}, D_k)$, not on the in-flight repair it interrupts; since in-flight repairs are uncommitted, interrupting one changes no committed state. Appending the new disruption to D_k thus yields the same problem instance as a fresh single-shot recovery from S_k against the enlarged D_k . Safety and termination follow from Proposition ?? applied to each single-shot recovery, and the FK bound from at most F such recoveries, each capped at K rounds. $\square$

Corollaries of the Authority-Separation Theorem (main paper, Section 3.1)

Corollary 1 (Solver Non-Authority). *A solver certificate proves only that a solver produced a plan under its own assumptions. It does not make the plan admissible. The plan remains a proposal until ATP admission accepts it under $\mathcal{C}$.*

Corollary 2 (Effective-State Soundness). *If StateView is derived only from non-compensated, non-superseded, dependency-valid committed records, then every StateView fact corresponds to an effective committed chain, and no compensated or superseded record appears as current truth.*

Corollary 3 (Dependency-Closed Compensation Safety). *If compensation is admitted only when it preserves dependency closure and effective-chain continuity, then no admitted compensation can leave an effective dependent orphaned or produce an effective state lacking a valid committed chain.*

Table S1: Generated-workflow failures and ATP mechanisms.

Failure	ATP mechanism
Invalid transition	deterministic admission
Unsafe compensation	dependency-closed compensation
Conflicting proposals	scoped proposal-conflict rejection
Stale assumptions	stale-world rejection
Evidence-destroying repair	evidence-preserving admission
Unbounded live obligation	active commitment lifecycle
Duplicate concurrent repair	idempotent recovery-event admission

S2 Expanded Failure Model

Mnemosyne addresses failure modes that ACID does not govern, because ACID constrains a unit of work only after it is admitted. We describe the principal modes below; Table S1 pairs each with the mechanism that rejects or contains it.

Invalid generated transition. An LLM proposes a workflow transition that violates a finite-state constraint. A conventional workflow engine may persist the action unless application code rejects it. ATP requires a deterministic validator to reject it before commit.

Compensation with effective dependents. A record is compensated after downstream records have depended on it. Raw log history still contains the record, but it is no longer effective. ATP disallows compensation that would orphan currently effective dependents or break the effective-state chain.

Conflicting proposals. Two agents propose different plans for the same tenant and entity. Both may be individually feasible. ATP rejects the active proposal set before commit because the proposals conflict over the same admission scope.

Stale-world proposal. A solver proposes a route assuming a deadline of 17:00, but an observed world snapshot reports the deadline is now 11:00. ATP rejects the proposal before commit because its assumptions no longer match observed facts.

Evidence-destroying repair. A coding agent observes that continuous integration is red and proposes to “repair” the repository by deleting the failing tests. A periodic checker may see the next CI run turn green and conclude the violation is gone, but the defect has not been repaired: the evidence has been destroyed. ATP requires admission to preserve the evidence that triggered repair unless the validator accepts that the repair addresses the underlying condition rather than suppressing its observable signal.

S3 Additional Safety Invariants

Beyond the safety invariants stated in the main paper, the runtime also enforces the following.

Invariant 1 (Effective-chain preservation). *A compensation is rejected if it would break the effective chain for an entity.*

Invariant 2 (Admitted commitment creation). *Creating an ACR is itself an admitted transition. The record must have bounded dependency scope, a discharge or expiry condition, a guard, a validator, and an idempotent creation key.*

Invariant 3 (Scoped repair). *A proposal generated by an ACR may touch only the dependency scope recorded by that ACR, unless a wider scope is explicitly admitted as an escalation.*

Table S2: Aggregate safety scoreboard across the nine benchmarks: each permissive baseline commits the targeted violation, while ATP commits zero and still admits valid operations.

Class	Benchmark	Baseline violations	ATP violations
Authority	AuthorityBench	35 invalid generated commits	0 invalid generated commits
Serial	SerialAdmissionBench	48 invalid commits and capacity underflow	0 invalid commits; serial-equivalent history
Evidence	EvidenceRepairBench	20 evidence-destroying repairs	0 evidence-destroying repairs
Obligation	ObligationBench	16 unauthorized wakeup mutations	0 unauthorized wakeup mutations
Projection	CompensationProjectionBench	7 StateView mismatches	0 StateView mismatches
Storage	StorageSubstrateBench	64 invalid storage commits and 1 StateView mismatch	0 invalid commits and 0 StateView mismatches
End-to-end	J1J4EndToEndBench	3 invalid commits and 3 StateView mismatches	0 invalid commits, 0 mismatches, 4 completed cases
Proposer	ProposerQualityBench	37 invalid direct commits	0 invalid commits across six proposer classes
Comparator	GuardrailComparator	6 ATP-specific hazards missed by workflow/saga guardrails	0 invalid commits; 4 valid commits admitted

Invariant 4 (Cross-episode traceability). *Rejected or compensated ACR proposals with matching failure signatures remain queryable as historical evidence for future proposal ranking, guard strengthening, or external causal-learning modules.*

S4 Benchmark Roster

The main paper’s evaluation (§6) summarizes the released benchmark suite compactly; the full roster is as follows. The six safety benchmarks check, respectively, that generated proposals stay non-authoritative until admitted (**AUTHORITYBENCH**); that concurrent proposals commit to a serial-equivalent history (**SERIALADMISSIONBENCH**); that a repair cannot discharge its own trigger by destroying evidence (**EVIDENCEREPAIRBENCH**); that commitment wakeups resume recovery without mutating domain truth (**OBLIGATIONBENCH**); that compensation and supersession preserve effective-state projection (**COMPENSATIONPROJECTIONBENCH**); and that storage-level uniqueness, idempotency, and transactional projection reject duplicate or malformed attempts without corrupting **StateView** (**STORAGESUBSTRATEBENCH**). **J1J4ENDTOENDBENCH** drives planning and recovery cases through the boundary, **PROPOSERQUALITYBENCH** varies proposer quality while tracking usefulness and the invalid-commit invariant, and **GUARDRAILCOMPARATORBENCH** compares ATP with a workflow/saga guardrail stack (schema validation, finite-state checks, idempotency keys, timers, local saga compensation, proposer self-checking) that lacks ATP-specific admission over stale-world assumptions, effective state, evidence preservation, obligation containment, dependency-closed compensation, and conflict scopes.

S5 Aggregate Safety Scoreboard

This is the full aggregate safety scoreboard summarized in the main-paper evaluation.

Table S3: RQ4 proposer-level result under ATP. Proposer quality changes acceptance rate, utility, and first-admission delay; invalid commits remain zero for every proposer class.

Proposer	Attempts	Admitted	Rejected	Accept. rate	Utility	First admission
No intelligence	10	1	9	0.10	1	8
Random	10	2	8	0.20	4	5
Rule-based	10	5	5	0.50	25	2
LLM-like	10	6	4	0.60	36	2
Solver-like	10	8	2	0.80	64	1
Adversarial	10	1	9	0.10	1	10

S6 Per-Proposer-Class Detail

This table gives the per-proposer-class detail behind the proposer-quality study of the main paper (the corresponding bar chart appears in the main paper).

S7 Concurrency, Contention, and Distributed Execution

Concurrency and contention. The serialization unit is the conflict scope, not the whole log: proposals over disjoint scopes (distinct tenant, entity, or resource) commit concurrently, so throughput scales with scope-level parallelism rather than a single global lock. Within one hot scope, the PostgreSQL substrate orders contenders through row-level locking and a unique constraint over (tenant, scope, idempotency key, recovery sequence position), so concurrent proposals either converge to one canonical committed transition or reject the conflicting losers, exactly the boundary of the Serial-Equivalent Generative Admission Theorem; MVCC snapshot isolation lets the **StateView** projection read effective state without blocking admission. A large disruption that wakes many obligations therefore degrades to serialized admission *on the affected scopes only*, trading latency for serial equivalence rather than risking an invalid interleaving, and rejected contenders retry against the updated **StateView** under the memoryless repair model (the Reactive Repair section of the main paper). Cross-scope independence is what keeps dense conflict-scope collisions from becoming a global bottleneck.

Distributed execution. The evaluation runs against a single durable store, which is the linearization point for committed truth: each tenant and conflict scope has one authoritative CTL, so the authority boundary is well defined within a store even when proposers, solvers, and orchestrators are distributed. External orchestration engines such as Temporal drive execution but never hold commit authority (Appendix S18); under network partitions or worker-node failure they re-drive activities at least once, and the operation idempotency key together with the outbox deduplication boundary make replays converge to a single committed transition and a single external effect rather than duplicating either. What we do not yet evaluate is multi-region or sharded operation, where committed truth would span several stores: maintaining serial-equivalent admission and outbox idempotency across asynchronous cross-region updates would require partitioning conflict scopes across stores or a shared consensus log, which, with production-scale load testing, we leave to future work.

S8 Per-Property Safety Results

This appendix gives the full per-class result tables for the six safety hazard classes summarized in RQ1 (the main-paper evaluation) and visualized in the safety-scoreboard figure of the main paper; each compares Mnemosyne against the deliberately permissive baseline(s) for that class. Table S4 reports **AUTHORITYBENCH**: a raw-append baseline and a self-validation baseline each commit 35 invalid generated transitions, while ATP commits zero and still admits the five valid proposals. Table S5 reports **SERIALADMISSIONBENCH**: under 80 concurrent proposals over a shared capacity object, the unserialized and weak-lock baselines commit invalid proposals and underflow capacity, while ATP commits only the 32 valid proposals through a serialized boundary and yields a serial-equivalent history. Table S6 reports **EVIDENCEREPAIRBENCH**: the naive-repair and no-rule workflow baselines each commit 20 evidence-destroying repairs, while ATP commits zero and still admits the eight valid repairs. Table S7 reports **OBLIGATIONBENCH**: the trigger-direct and timer-direct baselines commit 12 and 16 unauthorized wakeup mutations, while ATP routes every wakeup through ordinary admission and commits zero. Table S8 reports **COMPENSATIONPROJECTIONBENCH**: the projection baselines admit five invalid compensations and produce seven **StateView** mismatches, while ATP produces zero and still commits the two valid compensations. Table S9 reports **STORAGESUBSTRATEBENCH**: the unconstrained log commits 64 invalid storage attempts and one **StateView** mismatch, while the ATP storage path rejects all 64 with zero mismatches.

Table S4: Authority separation. Unsafe baselines commit invalid generated proposals; ATP rejects all invalid proposals while still admitting valid ones.

System	Invalid commits	Rejected invalid	Valid commits
Raw append	35	0	5
Self-validation	35	0	5
ATP / Mnemosyne	0	35	5

Table S5: Serial-equivalent admission under concurrency. Unsafe baselines commit invalid concurrent proposals and produce non-serial-equivalent state; ATP admits only valid proposals through a serialized transaction boundary. Column SEA denotes serial equivalence.

System	Committed	Rejected	Invalid	Underflow	SEA
Unserial. writes	80	0	48	1	no
Weak lock	68	12	36	1	no
ATP / Mnemosyne	32	48	0	0	yes

S9 Live-Proposer Pilots (RQ3): Detail and Representative Cases

This section records the full detail behind the main paper’s RQ3: the pilot configurations, the complete scorer outputs, the staged pipeline, and the representative cases referenced there. Both pilots are bounded: one task family (job-shop), four proposer packs (Claude, GPT, DeepSeek expert, DeepSeek instant), manual collection, and no claim of learning efficacy or family generality.

Table S6: Evidence-preserving repair. Unsafe baselines admit repairs that hide trigger evidence while leaving the underlying failure unresolved; ATP rejects all such repairs and still admits valid repairs.

System	Evidence-destroying	Rejected evid.-dest.	Valid repairs
Naive repair	20	0	4
Workflow w/o rule	20	0	8
ATP / Mnemosyne	0	20	8

Table S7: Obligation containment. Unsafe baselines allow wakeups to mutate committed domain state without ordinary admission; ATP allows wakeups to emit proposal packages but rejects unauthorized mutations before commit.

System	Unauthorized mutations	Proposal packages	Admitted	Rejected
Trigger direct	12	0	0	0
Timer direct write	16	0	0	0
ATP / Mnemosyne	0	24	4	20

S9.1 Static plan-entry pilot

The static pilot uses one Tier 6 E7 sequence, `T6-7e17ef0cc5f3`, under the full C+R+T stack. Four response packs contribute ten episodes each, giving forty handoff-derived cases; all forty validate with zero schema failures. Scorer outputs: safety gate passed (zero invalid commits, zero evidence-destroying repairs, zero orphaned dependents); mean horizon reward 0.90125; grounded admission rate 0.725. The staged pipeline is: collect and parse the packs; admit or reject each proposal; attach the kernel-admission trace; replay through the runtime evaluator; bridge admission labels to scorer actions; export the handoff bundle; import and score in REALM-Bench. Every stage preserves the authority boundary: live models propose, Mnemosyne admits or rejects, the external scorer reads boundary traces only.

S9.2 Dynamic mid-execution pilot

The dynamic pilot poses ten mid-execution disruption episodes over the job-shop E7 setting; each of the four packs answers all ten (40 responses of type **repair**, **reject**, or **observe**). Deterministic admission guards require committed operations to remain unchanged, committed evidence to be preserved, rollback scope to remain none or local, and repair radius to remain bounded. Scorer outputs: 24 admitted, 16 rejected (4 explicit safety rejections); safety gate passed; repeated-failure rate 0.0; mean horizon reward 0.925; grounded admission rate 0.6; observed mean time-to-correction 0.725 (40 observed, 0 censored).

S9.3 Representative cases

Admitted local repair. Claude, episode 1, proposed moving the uncommitted operation **J4-02** away from a failed **M2** machine while preserving committed operations **J2-01** and **J3-01**; the affected step was local, rollback scope none, evidence preserved, and the proposal passed admission.

Table S8: Effective-state and compensation safety. Unsafe projection baselines admit invalid compensations and project ineffective history as current truth; ATP rejects invalid compensations and preserves **StateView** as the projection of effective committed records only.

System	Invalid admitted	Orphaned deps.	Broken chains	StateView mismatches	Valid
Unsafe projection baselines	5	2	5	7	2
ATP / Mnemosyne	0	0	0	0	2

Table S9: Storage-substrate correctness. The unconstrained log baseline commits duplicate or malformed storage attempts; the ATP storage path rejects invalid attempts and preserves effective-state projection. The PostgreSQL-backed store is implemented and exercised by environment-gated live conformance tests; the default artifact path is PostgreSQL-free.

System	Subs.	Committed	Rejected	Invalid	Mismatch
Unconstr. log	mem	128	0	64	1
ATP storage	PostgreSQL	64	64	0	0

Safe rejection. DeepSeek expert, episode 3, proposed a repair touching two affected steps, J2-02 and J3-02, when the repair-radius limit was one; Mnemosyne rejected it with the structured reason **repair_radius_exceeded:2>1**. A bad proposal plus a correct pre-commit rejection is a safety success, not a system failure.

Refused global rollback. Claude, episode 9, itself rejected an unsafe global rollback request, recognizing that rollback would violate committed evidence for J2-01 and J3-01. The replay records this conservatively as a model-side rejection rather than a guard-derived safety rejection, because it was requested by the proposer rather than triggered by an admission guard; it illustrates that structured rejection reasons can serve as feedback for iterative replanning, which is out of scope for this paper.

S10 Coverage Audit: ATP and a Production Transaction-Processing Substrate

This appendix extends the RQ2 comparator (the RQ2 comparator table of the main paper) with a third layer: a production transaction-processing (TP) substrate such as PostgreSQL. ATP is not a replacement for production TP. PostgreSQL-style systems provide the durable substrate, with atomicity, isolation, durability, uniqueness, foreign keys, checks, triggers, and serializable execution. ATP is complementary: it governs whether a generated proposal should receive transaction authority before it becomes an ordinary transaction. The distinction matters because several agentic hazards are not storage-level violations unless application developers manually encode ATP-specific semantics.

Table S10 reports the benchmark’s coverage table. It is a deterministic coverage audit, not a throughput benchmark: each entry is a by-design assessment of which layer naturally owns the corresponding check.

The audit identifies a layer boundary rather than a weakness in PostgreSQL or other TP systems. Production TP systems are strong commit substrates, but they do not by themselves define which generated proposal should be granted transaction authority. ATP adds **StateView**-based admission, retained-evidence checks, dependency-closed compensation, non-authoritative ACR wake-

Table S10: Coverage audit: a production TP substrate versus workflow/saga guardrails versus ATP. Entries are by-design assessments of which layer owns each check, not measured throughput.

Hazard	PostgreSQL-style TP	Workflow/saga	ATP/Mnemosyne
Primary-key / unique duplicate	caught	caught	caught
Missing foreign-key dependency	caught	caught	caught
Finite-state invalid transition	app-only	caught	caught
Stale-world proposal	missed	missed	caught
Orphaning compensation over effective state	app-only	partial	caught
Evidence-destroying repair	missed	missed	caught
ACR direct domain mutation	missed	missed	caught
Generative conflict-scope collision	app-only	partial	caught
Duplicate side-effect intent	partial	partial	caught (staged, then rejected)

ups, and generative conflict scopes before a proposal becomes a transaction; once a proposal is admitted, ATP relies on a production TP substrate for atomic commit, durability, uniqueness, and isolation.

We implement this audit as `PRODUCTIONTPCOMPARATORBENCH`, a deterministic coverage benchmark rather than a throughput benchmark. The benchmark runs the same hazard set through three enforcement models: a PostgreSQL-style TP substrate, a workflow/saga guardrail layer, and ATP/Mnemosyne. Its purpose is to identify which layer owns each enforcement responsibility, not to claim that production TP systems are weak or that PostgreSQL cannot encode application-specific triggers.

The benchmark covers nine hazards. ATP/Mnemosyne catches all nine. The PostgreSQL-style TP substrate catches the two storage-level hazards natively, marks three as application-encoded, partially catches one, and misses three ATP-specific proposal-authority hazards. The workflow/saga guardrail layer catches three classical guardrail hazards, partially catches three, and misses three ATP-specific hazards.

The benchmark is reproduced by:

```
python benchmarks/realm/production_tp_comparator.py
python -m pytest -q tests/benchmarks/test_production_tp_comparator.py
```

It emits `summary.json`, `summary.csv`, and `report.md` under:

```
benchmarks/realm/reports/
production_tp_comparator/
```

The exact VLDB artifact snapshot is:

<https://github.com/eyuchang/Mnemosyne/tree/arxiv-atp-rq1-rq9b-r8-v2>

S11 RQ5 Infrastructure-Cost Audit: Full Method and Results

This section records the full method and result table behind the main paper’s RQ5 summary.

RQ2 establishes that ATP rejects the ATP-specific hazards in the mechanism-level workflow/saga comparator; RQ5 audits the cost visible in the implemented ATP infrastructure paths. We do not time the RQ2 oracle comparator as though it were the ATP runtime, and we do not claim a production load test. Instead, we run a repository infrastructure workload covering kernel admission, runtime admission, commit-boundary enforcement, effective-state projection, validation,

compensation, active recovery, and Temporal activity-boundary validation. Each repeat executes 20 ATP infrastructure paths, and we measure 30 repeats at worker counts 1, 4, and 8. The workflow/saga semantic comparator is still executed to preserve the RQ2 baseline, but its single semantic test is not treated as a throughput competitor for the 20-path ATP infrastructure workload.

Table S11: RQ5 artifact-level infrastructure-cost audit for ATP’s implemented safety paths. Each run executes 20 real repository paths covering admission, commit-boundary enforcement, effective-state projection, validation, compensation, active recovery, and Temporal activity-boundary validation. Latencies are p_{50}/p_{95} in milliseconds; throughput is infrastructure-test units per minute. All settings use 30 repeats.

Workers	Pass	E2E ms	Admission ms	Commit ms	Projection ms	Validation ms	PV overhead	Throughput units/min
1	30/30	5192.7 / 5266.5	0.040 / 17.235	2.574 / 8.620	0.023 / 1.731	0.001 / 0.057	5.98%	230.56
4	30/30	6356.6 / 6740.5	0.051 / 22.877	2.997 / 9.294	0.025 / 2.024	0.001 / 0.067	5.63%	709.10
8	30/30	11929.3 / 12172.0	0.100 / 36.731	5.120 / 12.979	0.030 / 3.601	0.001 / 0.094	4.96%	804.43

Table S11 reports the real-infrastructure result. All ATP infrastructure runs pass at all three worker counts. At one worker, the 20-path workload requires 5192.7/5266.5 ms end-to-end p_{50}/p_{95} , with admission latency 0.040/17.235 ms and commit latency 2.574/8.620 ms. At four workers, measured throughput rises to 709.10 infrastructure units per minute, with projection plus validation accounting for 5.63% of end-to-end runtime. At eight workers, measured throughput reaches 804.43 infrastructure units per minute; admission and commit tail latency increase, as expected under concurrent pytest execution, but projection plus validation remains bounded at 4.96% of end-to-end runtime. Thus ATP safety is not free, but the artifact exposes where its implemented infrastructure cost appears. Together, RQ2 and RQ5 report the safety result and the local artifact overhead audit; they do not establish production-scale throughput or a product-level comparison against a deployed workflow engine.

S12 R8 Deployment Smoke-Load Variability

R8 implements a deployable Mnemosyne service boundary with health, proposal-submission, State-View, and metrics endpoints. The purpose of the R8 smoke-load audit is not to claim production-scale throughput, but to check that the deployment service preserves ATP’s authority-separation rule under concurrent HTTP proposal submission: clients may submit proposals, but invalid proposals and explicit bypass attempts must not become committed truth.

Table S12 reports two consecutive local runs of the same deployment load script. Each worker setting submits 200 HTTP proposal requests. The workload contains 120 valid proposals and 80 invalid or bypass proposals. In both runs, every worker setting admits exactly 120 proposals, rejects exactly 80 proposals, and produces zero invalid commits. Thus the safety result is stable across repeated local runs.

The throughput measurements are intentionally treated as diagnostic rather than production evidence. The 16-worker setting shows substantial run-to-run throughput variability despite similar p_{50}/p_{95} request latency, indicating sensitivity to local client scheduling, socket backlog, and short-run wall-clock effects. We therefore use R8 only as deployment-boundary evidence: concurrent HTTP submission preserves the ATP admission boundary, while production load testing over durable PostgreSQL storage, Kubernetes workers, networked workflow services, and pool saturation remains future work.

Table S12: R8 local deployment smoke-load variability across two consecutive runs. Each setting submits 200 HTTP proposal requests to the R8 service. The stable result is safety: invalid commits remain zero in every run and worker setting. Throughput is local diagnostic evidence, not a production load-test claim.

Run	Workers	Accepted / Rejected	Invalid commits	Latency p_{50}/p_{95} ms	Throughput /min
1	1	120 / 80	0	0.281 / 0.381	119,717.96
1	4	120 / 80	0	0.871 / 1.515	243,307.15
1	8	120 / 80	0	1.744 / 2.892	245,481.60
1	16	120 / 80	0	3.398 / 6.226	245,801.74
2	1	120 / 80	0	0.250 / 0.380	173,746.85
2	4	120 / 80	0	0.822 / 1.307	271,099.39
2	8	120 / 80	0	1.743 / 3.136	257,312.64
2	16	120 / 80	0	3.047 / 5.396	11,784.96

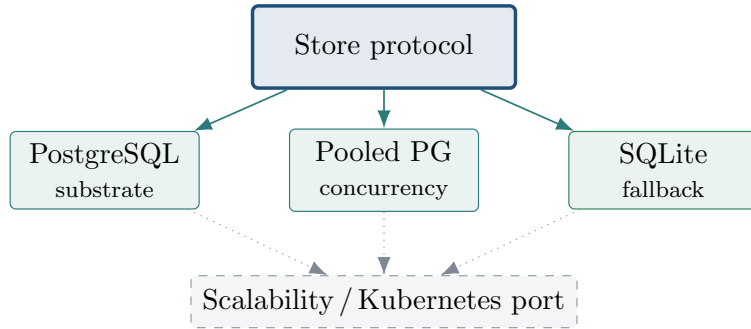


Figure S1: Storage substrate boundary. PostgreSQL (and pooled PostgreSQL for concurrency) is the substrate for all reported experiments; SQLite remains a zero-setup fallback for portability. The backing store is an implementation choice behind the store protocol: it affects neither ATP correctness semantics nor any reported number. The scalability and Kubernetes deployment phase changes deployment scale, not correctness.

S13 Mnemosyne Architecture and Runtime Data Structures

This appendix specifies the runtime data structures that realize the architecture of the Architecture section of the main paper at the level of record fields. The intent is to make the authority boundary inspectable: every object below is either committed truth, a projection of committed truth, a durable obligation, or an external boundary, and no proposer-side object can become truth except by passing admission.

Object taxonomy. Mnemosyne keeps four kinds of object apart, and logs only the last. An *event* is an immutable observed fact (for example, a provider confirmation or a disruption notice). A *candidate* is a proposed finite-state transition that an admitted commit may or may not produce. A *constraint evaluation* is a predicate that the admission validator checks. A *committed transition* is the durable record of an admitted transition: a CTL record. Events, constraints, and finite-state structure live outside the CTL; only an admitted, committed transition is a CTL record. A vetoed or rejected proposal leaves a queryable rejection or escalation entry, never a committed transition.

Table S13: CTL record fields. The identity and ordering fields fix committed truth; M and X are typed side-channels for recovery metadata and domain or causal evidence. The record is logically append-only: later transitions may compensate or supersede it, but never erase it.

Field	Symbol	Role
<code>rid</code>	r	Unique record identifier (primary key); the idempotency key when client-supplied.
<code>op_id</code>		Optional client idempotency key, used when <code>rid</code> is server-assigned.
<code>tx_group_id</code>		Saga group: the records that compensate together (indexed, not unique).
<code>workflow_id</code>		Session or instance identity (for example, a ride or trip).
<code>binding_id</code>		Role-binding identity (an abstract role bound to entities).
<code>eid</code>		Individual entity whose state changed.
<code>fsm</code>		Finite-state machine to which the edge belongs.
<code>version</code>	v	Monotonic version per (<code>eid</code> , <code>fsm</code>); one ordered history per entity.
<code>state_before</code>	s^-	Pre-state; the reversal target for compensation.
<code>state_after</code>	s^+	Post-state of the committed transition.
<code>action_type</code>	a	Named action (for example, pickup, reroute, compensate); disambiguates self-loops.
<code>triggers</code>	U	Proximate causes (event identifiers in the recovery-event log).
<code>dependencies</code>	D	Required preconditions: <code>rids</code> that must be committed and still effective.
<code>metadata</code>	M	Verdict, compensation policy, restore flag, status, supersession or compensation links, provenance.
<code>extension</code>	X	Schema-validated domain and causal-audit evidence plus temporal-spatial attributes.
<code>schema_id / schema_version</code>		Schema identity for safe replay across schema evolution.
<code>timestamp</code>	t	Commit time; <code>log_position</code> (below) provides ordering.

The committed-transition log record. A CTL record is the unit of committed truth. Its identity and ordering fields are fixed; its recovery metadata and domain evidence are carried in two typed side-channels, the metadata map M and the typed extension X , so that recovery and provenance fields can evolve without changing a transition’s identity. Table S13 lists the fields.

The storage layer enforces two uniqueness constraints that carry the safety load: `rid` is the primary key, so a retried commit cannot double-apply, and (`eid`, `fsm`, `version`) is unique, so each entity has exactly one ordered history. The saga group `tx_group_id` is indexed but deliberately *not* unique, so one saga spans many records. The metadata map M carries the recovery and audit fields, including `status` (one of `active`, `compensated`, or `superseded`), `restore` (whether the record is a clean restore point), and the `compensates` or `supersedes` links naming the `rids` a record reverses or replaces. The typed extension X carries domain attributes (location, ETA, deadline, route, assignment) and causal-audit evidence; X is schema-validated rather than free-form.

Identity and log position. Mnemosyne keeps three identities distinct rather than collapsing them into one: the individual entity `eid`, the role-binding `binding_id`, and the session or instance `workflow_id`. Conflating them breaks repeated participation (one entity acting twice), multi-party instances (one session with several participants), and reassignment (a resource re-bound to a new role). Independently of the wall-clock `timestamp`, the store assigns a monotonic `log_position` that provides total append ordering for replay and recovery; replay can proceed by per-entity `version` or by global `log_position`. The recovery-event log records the fine-grained events of repair and

Table S14: ACR fields. An ACR is a durable obligation in the CTL; its wakeup is a non-authoritative proposal that re-enters admission.

Field	Role
<code>id</code>	Identifier of the obligation record.
<code>txn</code>	Admitted transition that created the obligation.
<code>commitment</code>	The recorded commitment or assumption.
<code>scope</code>	Bounded dependency scope the resumed proposal may touch.
<code>trigger</code>	Condition whose firing makes the obligation ready.
<code>continuation</code>	Pointer to the continuation resumed on firing.
<code>guard</code>	Predicate gating whether a fired trigger may resume.
<code>validator</code>	Admission validator applied to the resulting proposal.
<code>compensation</code>	Compensation handler for the obligation.
<code>expiry</code>	Expiry or discharge condition.
<code>failure-key</code>	Failure signature for cross-episode traceability.
<code>status</code>	Lifecycle state (live ... discharged).

obligation execution (packages created, idempotency keys observed, sequence positions assigned, conflicts detected, outcomes accepted or rejected). In the PostgreSQL substrate, tenant-scoped uniqueness over event identifiers, idempotency keys, and recovery sequence positions provides the concrete concurrent-idempotency boundary.

StateView: the effective-state projection. `StateView` exposes current operational truth. It is reconstructed from the CTL by replaying only *effective* records: records that have not been compensated, superseded, or invalidated and whose dependency chains remain effective. Current state is derived, never stored. Beyond the bare finite-state value, a `StateView` for an `(eid, fsm)` pair folds the *X* attribute deltas forward (location, ETA, deadline, route, assignment) and carries the set of effective records that support it, so that admission and the reactive loop read where each entity currently is, not a static plan. The projection is materialized per `(eid, fsm)` and is a cache: replay from the CTL remains the source of truth. A cross-entity correction must re-project *every* affected entity, because refreshing only the issuing entity could leave a now-ineffective record lingering as current truth elsewhere.

Active commitment records. An ACR is a durable workflow obligation stored in the same committed substrate as ordinary transitions. It is created by an admitted step, and creating it is itself an admitted transition. An ACR records an assumption or commitment together with the fields in Table S14. When its trigger fires, the runtime may resume the continuation to construct a repair proposal, but the ACR itself cannot mutate committed state: its wakeup re-enters the ordinary admission gate. ACR status moves through values such as `live`, `fired`, `proposed`, `admitted`, `rejected`, `compensated`, `expired`, and `discharged`.

Inbox and outbox boundaries. External events enter through an inbox deduplication boundary; external side effects leave through an outbox boundary that stages provider calls with idempotency keys. The outbox is an intent log: provider execution remains outside committed truth until its result is observed and admitted as a subsequent transition. This prevents a domain commit from silently becoming an external effect, and prevents a duplicate external observation from creating a duplicate commit. Both boundaries sit on the commit side of the authority line but neither owns

truth: the inbox only deduplicates inbound observations, and the outbox only stages outbound intent.

S14 Admission, Commit, Logging, and Replay

This appendix gives the deterministic admission contract under $\mathcal{C}$ and the commit, logging, and replay sequence at the mechanism level. The contract is independent of proposer identity: an LLM, solver, runtime driver, benchmark adapter, or ACR wakeup may change the proposal package, but only the ordered admission steps can create committed truth.

The admission contract. Admission is the deterministic pre-commit procedure that decides whether a proposal may become a committed transition. It reads current effective state from `StateView` and retained evidence, never speculative or rejected proposal history, and applies the ordered checks below. Steps 1–8 are the executable contract used by the implementation and the RQ1–RQ6 experiments.

1. Parse the proposal package and reject malformed packages before commit processing begins.
2. Check tenant, entity, idempotency key, operation key, and declared conflict scope.
3. Read current `StateView` and retained evidence; do not validate against speculative or rejected proposal history.
4. Apply the deterministic constraints in $\mathcal{C}$: finite-state rules, policy rules, stale-world checks, dependency closure, and compensation safety.
5. If the proposal is a repair, require that the underlying failure is resolved under $\mathcal{C}$, or that the trigger evidence remains effective and queryable after the repair.
6. If the proposal is an ACR wakeup, require a proposal package and forbid direct mutation of the CTL or `StateView`.
7. Serialize or reject overlapping incompatible conflict scopes; admit only if the resulting transition is valid over current effective state.
8. Atomically append the admitted transition and recovery metadata to durable storage and update `StateView`; otherwise record a queryable rejection reason.

A solver certificate or LLM rationale is treated as evidence supplied to these checks, not as authority. The validator reads log-grounded effective state only: a finite-state edge is checked for legality with `action_type` disambiguating self-loops; each dependency is required to be committed *and* still effective (existence alone is insufficient); and application constraints are keyed by `(fsm, action_type)` so that, for example, a pickup may be required to follow the driver’s arrival as read from the driver’s effective state. The gate returns an admitted transition, a rejected proposal with structured reasons, or an escalation request for a wider scope.

Commit, logging, and projection. An admitted proposal commits through a single atomic step that also produces the projection update. The commit batch is the unit of multi-entity atomicity. In one transaction it: locks the affected entities in sorted key order, verifies the expected monotonic **version** for each (**eid**, **fsm**), re-checks that every dependency in D is committed and effective, appends all CTL rows, updates the synchronous **StateView** projection for every affected key, and stages any outbox intents. Three guarantees hold exactly at the write: idempotency (the **rid** primary key, with **op_id** as the client key when **rid** is server-assigned), ordering (the monotonic per-entity **version**), and dependency effectiveness (D committed and not since compensated or superseded). Only after this step does a row exist; a rejected proposal yields a queryable rejection, not a row.

Idempotency and retry safety. Because durable runtime drivers execute steps at least once, the commit path must be idempotent under retry. The **rid** primary key (or **op_id** when **rid** is server-assigned) makes a retried commit return the existing committed record rather than duplicate domain truth. This is why the idempotency key is part of the proposal package and is checked at Step 2 before any state mutation: a retried submission after a crash or timeout resolves to the same committed transition, so the commit batch is safe to re-drive.

Replay. The CTL is the source of committed truth, and current state is recovered by replay rather than read from a stored cursor. Replay of an (**eid**, **fsm**) history proceeds in order and self-checks consistency: each record's **state_before** must equal the prior record's **state_after**, and a mismatch is a replay error that signals log corruption. The bare replay yields the finite-state value; the full reconstruction is the **StateView** that additionally folds the X attribute deltas forward and retains the effective records supporting the state. Because compensation and supersession are themselves later admitted transitions, effective-state replay reproduces current truth without deleting history or moving a cursor: the effective index marks which records still count, and the projection excludes the rest.

S15 Disruption, Recovery, and Compensation

This appendix specifies recovery at the mechanism level. The central property is that recovery does not bypass admission: a disruption, a fired obligation, or a crash-recovery action produces a proposal that passes through the same gate as any generated action. Recovery decisions are made in the proposal and policy layers; the substrate supplies only the mechanism and the invariants.

Obligation wakeups do not bypass admission. When an ACR trigger fires, the runtime may resume its continuation and construct a repair proposal, but the wakeup carries no write authority. The resumed proposal declares the ACR's bounded dependency **scope** and re-enters the ordinary admission gate, where Step 6 requires that an ACR wakeup supply a proposal package and forbids direct mutation of the CTL or **StateView**. A proposal generated by an ACR may touch only the dependency scope recorded by that ACR, unless a wider scope is explicitly admitted as an escalation. Thus a fired obligation can only return the system to the proposal-admission path; it cannot install committed truth on its own. Rejected or compensated ACR proposals with matching failure signatures remain queryable as historical evidence for future ranking or guard strengthening, which is how memory becomes active without becoming authoritative.

Table S15: Recovery moves. The action’s class against the current governed-invariant set selects the move; the substrate supplies the mechanism.

Move	When	Mechanism
undo	An exact inverse exists (reversible).	Apply the inverse; the entity returns to s^- .
compensate	No inverse, but governed invariants are restorable within a validity window (compensable).	Append a compensating transition; history is preserved.
escalate	Neither holds, or local repair cannot contract (irreversible).	Veto, or hand to a human task; no committed transition is written.

Three recovery moves. The class of an action against the full governed-invariant set, re-checked at commit time, selects the recovery move; the substrate supplies the mechanism and the policy layer supplies the timing. Table S15 states the three moves. A restore (return to a clean checkpoint) is the bulk form of an undo: it resets state but, unlike an exact undo, does not reverse external effects, so it is paired with compensation.

Dependency-closed compensation and supersession. Compensation is represented as a new admitted transition, never as physical deletion, and the whole saga group identified by `tx_group_id` unwinds together. The group is compensated in reverse topological order over the group’s dependency subgraph, not merely reverse insertion order, so a leg is never compensated before the legs that depend on it. Admission enforces dependency-closed compensation through the recovery invariants: a compensation is rejected if it would leave an effective record depending on an ineffective record (no orphaned effective dependents), and a compensation is rejected if it would break the effective chain for an entity (effective-chain preservation). Supersession is the related operation by which a later admitted transition replaces an earlier one through the `supersedes` link in M ; like compensation, it changes effectiveness without erasing history.

Effective-chain preservation under cross-entity correction. Compensation and supersession never delete history; they only change which records are effective. Because the projection is materialized per `(eid, fsm)`, a correction that touches several entities must drop the corrected record from current truth on *every* entity it touches, not only the entity that issued the correction. Refreshing only the issuing entity would let a now-ineffective record linger as current truth elsewhere. Replay from the CTL therefore remains the source of truth, and the per-entity projection is rebuilt for all affected keys after a correction.

Evidence-preserving repair. A memory-initiated repair must not discharge its own trigger by deleting, overwriting, compensating, or obscuring the evidence that justified it. Admission permits a repair to alter trigger evidence only if it verifies, under the retained evidence and validator rules, that the proposed repair addresses the triggering condition rather than merely suppressing its observable signal (Step 5). Concretely: let a repair proposal be triggered by evidence E of a failure condition F . Admission admits the repair only if either F is resolved under $\mathcal{C}$, or E remains effective and queryable after the repair. As a result, no admitted repair can clear its own trigger by destroying the evidence that caused it, and a repeated failure remains visible through its failure signature for later analysis.

Crash recovery as invariants in reverse. A transactional fault, such as a crash mid-leg, is handled with the same mechanisms. Recovery locates the most recent record marked as a clean restore point, appends compensating transitions for the saga groups committed after it in reverse topological order (marking them **compensated**), and resumes the reactive loop from the resulting effective state. Nothing is deleted and no cursor moves: the original records remain for audit, and effective-state replay yields current truth because each compensation is itself a later committed transition. Recovery is thus the ordinary commit invariants (idempotency, monotonic versions, dependency effectiveness) run in reverse under a policy.

S16 Staged Implementation (R0–R8) and Reproducibility

This appendix records the staged build of Mnemosyne and the reproducibility path. The build follows a small-to-large discipline: a visible, auditable correctness kernel is established before scale-out infrastructure is connected, so that the authority boundary (the central claim) is not obscured by an external engine or solver. Table S16 lists stages R0 through R8 and their status. The deployment service boundary (R8) is now complete: a deployable service with health, proposal-submission, **StateView**, and metrics endpoints is implemented and audited for authority separation under concurrent HTTP submission (Appendix S12).

Default reproducibility path. The default reproducibility path is PostgreSQL-free and deterministic. It runs the nine executable safety experiment files directly with **pytest**; the expected result is **9 passed**. These nine experiments correspond to the safety properties evaluated in RQ1–RQ5 and the mechanism-level comparator: authority separation, serial-equivalent admission, evidence-preserving repair, obligation containment, effective-state and compensation safety, storage-substrate correctness, the J1–J4 end-to-end execution, proposer-quality safety invariance, and the workflow/saga guardrail comparison. A broader safe command additionally runs **tests/core**, **tests/apps**, and **tests/benchmarks**. Reviewers should not run all of **tests / experiments** as a blanket command, because older exploratory imports outside the ATP artifact path are intentionally excluded from the reproducibility suite.

Infrastructure-cost benchmark (RQ5). The cost audit is run as a separate infrastructure-cost benchmark over selected real infrastructure tests rather than as a violation-suppression experiment. It reports end-to-end, admission, and commit latency, projection-plus-validation overhead, and throughput at one, four, and eight workers; projection plus validation remains bounded at under 6% of end-to-end runtime. The benchmark script that generates the RQ5 table is included in the artifact alongside **ARTIFACT_EVALUATION.md**, which lists the exact safety validation command and expected reports.

Optional live PostgreSQL path. The live PostgreSQL path for the storage-substrate experiment is optional and environment-gated by the **MNEMOSYNE_POSTGRES_DATABASE_URL** variable, with an optional connection-pool dependency. When supplied, it validates recovery-event append and list operations, canonical duplicate-idempotency behavior, clean sequence-conflict handling, and pooled connection use. The artifact claim does not require a live database: default CI remains PostgreSQL-free and pool-dependency-free, so the **9 passed** result reproduces without any database setup.

Table S16: Staged implementation R0–R8. Each stage is independently runnable and testable. The correctness kernel and its safety boundary are completed before deployment scale-out; R8 changes deployment scale, not the semantics proved in the main text.

Stage	Delivers	Detail	Status
R0	Substrate kernel	CTL schema, append-only commit, monotonic per-entity versioning, PostgreSQL store, <code>log_position</code> ordering.	Complete
R1	Effective-state projection	<code>StateView</code> reconstruction from effective records; full and effective entity histories.	Complete
R2	Deterministic admission under $\mathcal{C}$	Isolated validator: finite-state legality, dependency effectiveness, conflict scope, stale-world checks; structured rejection reasons.	Complete
R3	Compensation and recovery	Dependency-closed compensation, supersession, restore points, fail-closed compensation invariants; recovery-event log.	Complete
R4	Idempotency and boundaries	<code>op_id</code> logical idempotency, inbox deduplication, outbox intent boundary, multi-entity atomic commit batch.	Complete
R5	Active commitment records	ACR creation as an admitted transition, non-authoritative wakeups, scoped repair, failure signatures.	Complete
R6	Benchmark and proposer integration	J1–J4 end-to-end harness; proposer-quality safety-invariance path; mechanism-level workflow/saga guardrail comparator.	Complete
R7	Storage substrate and cost audit	Optional environment-gated PostgreSQL and pooled-PostgreSQL paths behind the store protocol; RQ5 infrastructure-cost audit.	Complete
R8	Deployable service boundary	HTTP service with health, proposal-submission, <code>StateView</code> , and metrics endpoints; authority separation audited under concurrent submission.	Complete

S17 Structure and Scalability of the Constraint Set $\mathcal{C}$

A natural objection is that ATP merely moves the correctness burden from the agent prompt into the validator: if every semantic rule must be hand-coded, $\mathcal{C}$ does not scale. In the implementation most of $\mathcal{C}$ is not hand-coded per case. Its rules fall into three classes (Table S17).

1. **Structurally derived.** Generated mechanically from declarations the application already maintains. Finite-state legality is derived from per-entity transition table keyed by `action_type`; dependency effectiveness and compensation closure are derived from the effective-chain over CTL; idempotency and conflict scope are derived from the record’s identity triad. No per-rule code is written.
2. **Declarative predicates.** Declared once as data, not procedures, and checked by a generic evaluator: schema and type constraints, required-evidence handles, world-fact freshness keys, and conflict-scope keys.
3. **Application validators.** The only genuinely hand-written part: domain semantics that cannot be derived or declared (for example, a capacity rule specific to one operation). In the artifact these are a small minority of the constraint surface.

A proposal package is admissible when it satisfies the conjunction of all applicable constraints,

Table S17: Taxonomy of the constraint set $\mathcal{C}$: most rules are derived or declared, not hand-coded per case.

Class	How declared	Examples
Structurally derived	generated from FSM table, effective-chain, identity triad	finite-state legality, dependency effectiveness, compensation closure, idempotency, conflict scope
Declarative predicates	declared as data, evaluated generically	schema and type, required evidence, world-fact freshness
Application validators	hand-written per operation	domain capacity and business rules

which in grammar form is:

```

admit(p)      := structural(p) AND declarative(p) AND validators(p)
structural(p)  := fsm_legal(p) AND deps_effective(p)
               AND compensation_closed(p) AND idempotent(p)
               AND scope_consistent(p)
declarative(p) := schema_ok(p) AND evidence_present(p)
               AND world_fresh(p)
validators(p)  := conjunction of application-declared predicates

```

Because the structural and declarative classes are generated from declarations the application already keeps (entity FSMs, the dependency model, the schema), adding a new operation typically extends $\mathcal{C}$ by declaration rather than by new validator code. That is what keeps the admission boundary scalable as an application grows, and it bounds the hand-written surface to genuine domain semantics.

S18 Workflow-Engine and Storage-Substrate Selection

This appendix records the substrate selection and its rationale. The governing principle is that the substrate is non-authoritative: a workflow engine or storage backend may change deployment scale, availability, or throughput, but it must preserve the ATP authority boundary. Committed truth is produced only by admitted transitions, regardless of which engine orchestrates the work or which store persists the log.

Why the substrate is non-authoritative. Mnemosyne separates orchestration from authority. Runtime drivers schedule work, fire timers, retry steps, and wake obligations, but they do not own committed truth: a driver may detect that an ACR is ready, call a proposal provider, submit a package, and record a deterministic summary, yet it may not directly mutate domain truth. Keeping the source of truth in the CTL rather than in engine-internal state is a deliberate design fork. Letting the engine own state would tie the design to one engine and weaken the validator’s evidence story, because admission reads the effective-state projection rather than engine-private execution history. Holding the authority boundary in the admission gate is what lets a deployment substrate change without changing the correctness semantics.

Two-tier, PostgreSQL-anchored recommendation. The deciding requirement is a true ACID store with snapshot isolation, which PostgreSQL provides natively through MVCC. The recommendation is therefore a PostgreSQL-anchored two-tier substrate: a Postgres-backed durable-execution

Table S18: Durable-execution and orchestration engines considered. Each is an interoperability or deployment target behind an engine-agnostic interface; none owns the ATP authority boundary, which remains in the admission gate.

Engine	Model	Role for Mnemosyne
PostgreSQL	ACID store with MVCC snapshot isolation	Anchor substrate: hosts CTL, projection, recovery log.
SQLite	Embedded ACID store	Zero-setup fallback for local development and portability.
Temporal	Durable execution, code-defined sagas	Orchestration and scale escape hatch; non-authoritative.
Cadence	Durable execution (Temporal predecessor)	Orchestration interop target; non-authoritative.
Argo Workflows	Kubernetes DAG execution	Deployment and export target, not a transaction engine.

and transaction core that hosts the CTL, the effective-state projection, and the recovery-event log in one store, with a compute fan-out tier layered beneath it for concurrent proposal and probe execution. The durable tier supplies transactional bookkeeping and a measurable commit-failure rate; the compute tier supplies node-count scale. In the reference implementation the store protocol runs on PostgreSQL (and pooled PostgreSQL for concurrency) as the substrate for all reported experiments, with SQLite retained as a zero-setup fallback for local development and portability (Figure S1); the backing store affects neither correctness semantics nor any reported number. The scalability and Kubernetes deployment phase is a deployment concern that may add containerized services, durable-workflow workers, orchestration, observability, and load tests; it changes deployment scale, not ATP correctness semantics.

Durable-execution survey: interop targets, not authority. The 2025–2026 durable-execution category provides persistence, retries, and exactly-once or ACID semantics as infrastructure, and Mnemosyne builds the CTL and the ATP layer on top of such an engine rather than reimplementing logging or recovery primitives. Table S18 summarizes the engines considered. They are treated as interoperability or deployment targets behind an engine-agnostic interface, not as the authority for committed truth: durable workflow engines such as Temporal and Cadence orchestrate execution but are non-authoritative, and Argo Workflows is retained as a Kubernetes deployment and export target rather than a transaction engine. Checkpoint-persistence agent frameworks are explicitly excluded as the transactional substrate, because checkpoint persistence is not durable execution with exactly-once or ACID guarantees; they may serve only as optional agent-authoring layers above a real durable engine.

Engine portability. The durable-execution core sits behind an engine-agnostic interface so that the choice of orchestrator is a contained substitution. The Temporal-style boundary makes the contract concrete: the workflow layer is deterministic and side-effect free (timers, timeouts, retries, signals), while the activity layer performs the side effects (validate the commit batch, build the CTL records, commit, reproject `StateView`, stage outbox). The workflow orchestrates and the activity commits, but the store and CTL own truth. Because activities execute at least once under a retry policy, the `op_id` idempotency key makes a retried activity return the existing committed record rather than duplicate domain truth. This portability is why the same authority boundary holds whether the orchestrator is a local driver or a durable workflow engine, and why a change of engine

does not change the safety properties proved in the main text.

S19 Worked Proposal Examples

This appendix makes the *proposal is not truth* boundary concrete. The examples use compact JSON-like records to show what crosses the proposal boundary and what the admission gate does before anything becomes committed truth. They are illustrative but match the implementation contract used throughout the artifact: a proposal package carries a tenant, entity, operation, dependency set, world assumptions, conflict scope, evidence handles, and an idempotency key. Admission evaluates the package against the current effective `StateView`, retained evidence, and the declared constraint set $\mathcal{C}$. Two axes matter independently: whether the proposal is *well formed*, and whether the committed *output* it would produce is $\mathcal{C}$ -valid.

S19.1 Valid Proposal

A valid proposal may come from an LLM, solver, workflow driver, or active commitment wakeup. The proposer supplies a candidate action, but the candidate stays non-authoritative until admission accepts it.

```
{
  "proposal_id": "p_valid_hotel_001",
  "tenant": "t1",
  "entity": "trip_17",
  "operation": "book_hotel",
  "depends_on": ["flight_f1"],
  "world_assumptions": {"flight_f1_status": "confirmed"},
  "conflict_scope": "t1/trip_17/lodging",
  "evidence": ["user_request_e1"],
  "idempotency_key": "book_hotel_trip17_once"
}
```

Admission result. Accepted. The referenced flight remains effective in `StateView`, the world assumption is current, no incompatible proposal occupies the lodging conflict scope, and the idempotency key is fresh.

Committed effect. The admitted hotel-booking transition is appended to the committed-transition log (CTL); `StateView` projects the booking as effective current truth; a confirmation message may be staged in the outbox under its own idempotency key.

S19.2 Invalid Proposal

The next proposal is syntactically well formed but semantically stale. A classical parser would accept the record; ATP rejects it before commit because its assumptions disagree with observed effective state.

```
{
  "proposal_id": "p_stale_hotel_002",
  "tenant": "t1",
  "entity": "trip_17",
  "operation": "book_hotel",
```

```

    "depends_on": ["flight_f2"],
    "world_assumptions": {"flight_f2_status": "confirmed"},
    "observed_world": {"flight_f2_status": "cancelled"},
    "conflict_scope": "t1/trip_17/lodging",
    "evidence": ["user_request_e1"],
    "idempotency_key": "book_hotel_trip17_once"
}

```

Admission result. Rejected. The proposal depends on a flight that is no longer effective under the admitted observations. The rejection is recorded with a durable reason, but no domain transition is appended.

Committed effect. CTL is unchanged with respect to domain truth, *StateView* is unchanged, and no external side effect is staged.

S19.3 Valid Output After a Valid Proposal

A valid admitted proposal may create a valid outbox intent. The outbox record is not the external effect itself; it is an admitted intent to perform the effect under an idempotency key.

```

{
  "proposal_id": "p_notify_after_booking_003",
  "tenant": "t1",
  "entity": "trip_17",
  "operation": "stage_outbox_notification",
  "depends_on": ["hotel_booking_b1"],
  "message_type": "booking_confirmation",
  "recipient": "user_u1",
  "conflict_scope": "t1/trip_17/notification",
  "idempotency_key": "notify_trip17_booking_once"
}

```

Admission result. Accepted. The booking dependency is an effective committed fact, the outbox idempotency key has not been consumed, and no incompatible notification intent exists for the same scope.

Committed effect. CTL records the admitted outbox intent; *StateView* exposes the notification as staged; the provider call stays outside committed truth until its result is observed and admitted as a later transition.

S19.4 Invalid Output After a Valid Proposal

Even after a valid domain proposal, a later output proposal can be invalid. Here an agent tries to charge a card twice by changing only the provider-level idempotency key while targeting the same committed payment obligation.

```

{
  "proposal_id": "p_duplicate_charge_004",
  "tenant": "t1",
  "entity": "trip_17",
  "operation": "stage_outbox_payment",
  "depends_on": ["hotel_booking_b1"],

```

```

    "payment_obligation": "hotel_booking_b1/deposit",
    "amount": "250.00",
    "conflict_scope": "t1/trip_17/payment/deposit",
    "idempotency_key": "pay_deposit_trip17_b1_once",
    "provider_idempotency_key": "new_key_but_same_obligation"
}

```

Admission result. Rejected (or escalated for reconciliation). The proposal targets an already staged or satisfied payment obligation under the same semantic conflict scope and obligation idempotency key. A changed *provider* key is evidence, not authority to create a duplicate external effect.

Committed effect. No duplicate outbox payment intent becomes current truth, and the rejected package remains queryable as evidence for audit or guard strengthening. Only the first and third proposals commit; the second and fourth are rejected with a durable reason, so committed-state correctness is independent of how the proposal was generated.

S20 An End-to-End Transaction Trace

This appendix traces one disruption-recovery path through ATP, separating five objects that agentic systems often conflate: input observation, proposal package, admission decision, committed log record, and external side-effect intent. It continues the trip scenario of Appendix S19.

S20.1 Input Observation

```

flight_cancelled(flight_f1)
source_event_id = flight_provider_event_e123
observed_at      = 2026-06-30T10:15:00Z

```

The input observation is evidence. It is not, by itself, a domain repair, and it grants no write authority to the proposer or the runtime driver.

S20.2 Proposal Package

A planner or active commitment wakeup proposes a repair package:

```

{
  "proposal_id": "p_repair_trip_017",
  "tenant": "t1",
  "entity": "trip_17",
  "operation": "repair_trip_after_flight_cancellation",
  "depends_on": ["flight_f1", "hotel_h1"],
  "evidence": ["flight_provider_event_e123"],
  "conflict_scope": "t1/trip_17",
  "proposed_steps": [
    "compensate_hotel(hotel_h1)",
    "search_alternate_flight(trip_17)",
    "notify_user(user_u1)"
  ],
  "idempotency_key": "repair_trip17_after_flight_f1_once"
}

```


The proposal is not committed truth; it is a candidate package supplied to the admission gate.

S20.3 Admission

Admission evaluates the package against $\mathcal{C}$ and the current effective state:

checks:

```
dependency closure:    hotel_h1 depends on flight_f1
stale-world facts:     flight_f1 is observed cancelled
compensation safety:   no effective downstream record is orphaned
evidence preservation: flight_provider_event_e123 stays queryable
conflict scope:        no active repair owns t1/trip_17
idempotency:           repair_trip17_after_flight_f1_once is fresh
```

If all checks pass, the proposal is admitted; if any fails, the rejection and its reason remain durable, but the proposed repair does not become domain truth.

S20.4 Committed-Transition Log

An admitted repair appends records such as:

CTL append:

```
compensation_requested(hotel_h1)
repair_proposal_admitted(trip_17)
ACR_updated(trip_repair_obligation)
recovery_metadata(...)
```

CTL is the source of committed truth. Compensation is logical: the earlier hotel record is not erased; it becomes ineffective only through admitted compensating transitions and effective-state projection.

S20.5 StateView

StateView projects current effective truth from CTL:

StateView:

```
flight_f1           = cancelled
hotel_h1            = compensation_pending
trip_17             = repair_in_progress
trip_repair_obligation = live
```

StateView is not raw history: it excludes compensated, superseded, or otherwise ineffective records while preserving full CTL history for audit.

S20.6 Outbox

External actions are staged rather than executed as hidden side effects of the planner:

outbox:

```
notify_user(user_u1)
idempotency_key = notify_trip17_repair_once
status          = staged
```

An outbox intent is not the external effect itself. The provider call and its result stay outside committed truth until observed and admitted as later evidence, which prevents a generated proposal from silently turning into an irreversible external mutation.

S20.7 Boundary Summary

Input observation is evidence, not committed repair. A proposal is candidate intent, not truth. An outbox intent is a staged effect, not external truth. Only admitted CTL records define committed truth, and `StateView` is the effective projection used for future admission.